

Exception Test

¿Cuál es el resultado del siguiente código?

```
public static void main(String[] args) {
    Person person = new Person();
    person.sayHow(getHow());
    Person employee = new Employee();
    employee.sayHow(getHow());
    User user = employee;
    user.sayHow(getHow());
}

private static String getHow()
{
    return null;
}

interface How {
    public default void sayHow(String how) throws RuntimeException {
        sayHow(how, 1);
        try {
            if (null == how) throw new RuntimeException("again");
        } catch (RuntimeException rte) {
        }
    }

    public void sayHow(String how, int n);
}

class User implements How {
    @Override
    public void sayHow(String how, int n) {
        try {
            if (null == how) throw new RuntimeException("again user");
        } catch (Exception rte) {
            System.out.println("user");
        }
    }
}

class Person extends User {
    public void start() {
        sayHow(null);
    }

    @Override
    public void sayHow(String how, int n) {
        try {
            if (null == how) throw new RuntimeException("again employee");
        } catch (RuntimeException rte) {
            System.out.println("person");
        }
    }
}

class Employee extends Person {
    public void start() {
        sayHow(null);
    }

    @Override
    public void sayHow(String how, int n) {
        try {
            if (null == how) throw new RuntimeException("again employee");
        } catch (RuntimeException rte) {
            System.out.println("employee");
        }
    }
}
```

}

Elige una opción:

- a) employee, person, employee
- b) user, employee, person
- c) user, employee, employee
- d) person, employee, employee
- e) user, person, employee, employee

Java Executor Service & Callable

Considera el siguiente código:

```
class BackToTheFuture
{
    private Double whats InFuture
    (ExecutorService executor, Callable<Double> task)
    {
        // Lines of code will be placed here
    }
}
```

¿Cuál de las siguientes opciones, al insertarse en lugar de *//lines of code will be placed here*, would utilizaría ExecutorService con una tarea Callable para devolver los resultados deseados?

Elige una opción:

- a)

```
try {
    return executor.submit(task).get();
} catch (Exception ex) {
    return null;
}
```
- b)

```
try {
    return executor.submit(task);
} catch (Exception ex) {
    return null;
}
```
- c)

```
try {
    return executor.execute(task);
} catch (Exception ex) {
    return null;
}
```
- d) ExecutorService cannot invoke Callable interface objects.

Scoring System

Escribe una consulta para imprimir el ID y el NOMBRE de cada uno de los tres estudiantes con las puntuaciones más altas. La idea es imprimir los registros en orden descendente según la puntuación. Para puntuaciones coincidentes, ordena por ID en orden ascendente.

STUDENT		
Name	Type	Description
ID	Integer	unique ID, the primary key
NAME	String	student name
SCORE	Float	the Math score of the student

Sample Input

STUDENT		
ID	NAME	SCORE
1	Bob	50
2	John	65.5
3	Harry	45
4	Dick	85
5	Dev	25
6	Sid	98
7	Tom	90
8	Julia	70.5
9	Erica	81
10	Jerry	85

Ejemplo de salida:

```
6 Sid
7 Tom
4 Dick
```

Explicación. Los estudiantes están ordenados en orden descendente según sus puntuaciones en matemáticas, seguidos del orden ascendente de sus números de identificación, como se muestra a continuación:

Sid > Tom > Dick > Jerry > Erica > Julia > John > Bob > Harry > Dev

Las puntuaciones de Dick y Jerry fueron las mismas, por lo que se muestran en orden de ID.

NOTA. Usar cualquier gestor de Base de Datos de manera libre (TSQL para SQL Server, PLSQL para Oracle, PostgreSQL o MySQL/MariaDb, etc).

Clases estáticas

Resolver el siguiente ejercicio que se plantea a continuación. En un código fuente de Java, realizar lo siguiente:

1. **Método principal.** Crea una clase llamada TransactionAnalyzer con un método estático llamado analyzeTransactions(int[] transactions).
2. **Requerimiento del método.** Dado un arreglo de enteros llamado transactions, este método debe:
 - Calcular y retornar la suma total de todas las transacciones.

- Identificar y retornar la transacción más alta.
- Contar cuantas transacciones son superiores a \$1,000.00 y retornar este número.
- Retornar una lista de las transacciones únicas, números **NO REPETIDOS**, ordenadas de forma ascendente.

Java Classes

¿Cuál de las siguientes afirmaciones es verdadera?

Elige una opción:

- a) Las clases miembro no estáticas deben tener accesibilidad predeterminada o pública.
- b) Todas las clases anidadas pueden declarar clases miembro estáticas.
- c) Los métodos de todas las clases anidadas pueden declararse como estáticos.
- d) Las clases miembro estáticas pueden contener métodos no estáticos.

Static Analysis 3

¿Cuál es el resultado en el código de Piratetalk cuando se ejecuta con este comando?

```
$ java Piratetalk -DargProp="dog,"
```

```
import java.util.*;
public class Piratetalk
{
    Properties p = System.getProperties();
    p.setProperty("pirate", "scurvy");
    String s = p.getProperty("argProp") + " ";
    s += p.getProperty("pirate");
    System.out.println(s);
}
```

Elige una opción:

- a) dog, scurvy
- b) null, scurvy
- c) scurvy dog
- d) scurvy null

Spring Boot: API de información de pistas

Desarrolla una API REST que comparta detalles sobre pistas musicales para una plataforma de streaming de música. Esta API puede recibir, mostrar y consultar datos como el título de la pista, el álbum, la descripción y el número de reproducciones. El trabajo se evalúa en función de la precisión y la eficiencia con que la aplicación maneja y procesa estos datos.

Los datos de las pistas musicales se representan como un objeto JSON, donde cada objeto describe diversas propiedades de la pista. Se incluyen las siguientes propiedades:

- **id:** El identificador único entero del objeto (Long).
- **title:** Título de la pista (String).
- **Nombre del álbum:** El nombre del álbum correspondiente a la pista (String).
- **releaseDate:** La fecha de lanzamiento de la pista, con formato AAAA-MM-DD (Fecha).

- **playCount:** Un valor entero que indica cuántas veces un usuario ha reproducido esta pista (Entero).

Ejemplo de un objeto JSON con datos de una pista musical:

```
{
  "title": "Lost in Echoes",
  "albumName": "Echoes of the Unknown",
  "releaseDate": "2021-07-15",
  "playCount": 5000
}
```

El servicio REST debe exponer el endpoint `/music/platform/v1/tracks`, lo que permite gestionar los registros de pistas musicales de la siguiente manera:

Solicitud POST a `/music/platform/v1/tracks`:

- Crea un nuevo registro de pista musical.
- Espera un objeto de datos de pista de música válido como su carga útil, excluyendo la propiedad `id`.
- El servicio asigna un identificador único de tipo `long` al objeto añadido. • La respuesta incluye el registro creado con su ID único, y el código de respuesta es 201.

Solicitud GET a `/music/platform/v1/tracks`:

- Responde con una lista de todos los registros de pistas musicales y un código de respuesta de 200.

Solicitud DELETE a `/music/platform/v1/tracks/{trackId}`:

- Elimina el registro con el ID de pista especificado si existe en la base de datos.

Solicitud GET a `/music/platform/v1/tracks/sorted`:

- Proporciona una lista de registros de pistas musicales ordenados por título.
- El código de respuesta es 200.
- El sistema ordena por defecto por título en orden ascendente.

NOTA. Asegúrese de que el proyecto cumpla con todos los criterios de los casos de prueba al ejecutar las pruebas rspec proporcionadas. Utiliza la base de datos H2 por defecto. Comience implementando la solicitud POST a `music/platform/v1/tracks`, ya que para probar otros métodos es necesario que la funcionalidad POST funcione correctamente.

Ejemplos de solicitudes y respuestas

Solicitud POST a `/music/platform/v1/tracks`

Cuerpo de la solicitud:

```
{
  "title": "Lost in Echoes",
  "albumName": "Echoes of the Unknown",
  "releaseDate": "2021-07-15",
  "playCount": 5000
}
```

El código de respuesta es 201, y al convertirlo a JSON, el cuerpo de la respuesta es:

```
{
  "id":1,
  "title":"Lost in Echoes",
  "albumName":"Echoes of the Unknown",
  "releaseDate":"2021-07-15",
  "playCount":5000
}
```

Esto agrega un nuevo objeto a la colección con las propiedades dadas y el ID 1.

DESCRIPCIÓN DE LA PREGUNTA

Solicitud GET a `/music/platform/v1/tracks`. El código de respuesta es 200, y al convertirlo a JSON, el cuerpo de la respuesta, suponiendo que estos objetos están en la colección, es el siguiente: A continuación:

```
[
  {
    "id":1,
    "title":"Lost in Echoes",
    "albumName":"Echoes of the Unknown",
    "releaseDate":"2021-07-15",
    "playCount":5000
  },
  { "id":2,
    "title":"Bravos",
    "albumName":"Unknown",
    "releaseDate":"2021-07-16",
    "playCount":100
  }
]
```

Solicitud DELETE a `/music/platform/v1/tracks/1`. Suponiendo que el objeto con ID 1 existe, el código de respuesta es 204 y el cuerpo de la respuesta está vacío. Solicitud GET a `/music/platform/v1/tracks/sorted` El código de respuesta es 200 y, al convertirlo a JSON, el cuerpo de la respuesta —la colección devuelta ordenada por título en orden ascendente— es el siguiente:

```
[
  {
    "id":1,
    "title":"Abacus",
    "albumName":"Echoes of the Unknown",
    "releaseDate":"2021-07-15",
    "playCount":5000
  },
  {
    "id":2,
    "title":"Bravos",
    "albumName":"Unknown",
    "releaseDate":"2021-07-16",
    "playCount":100
  }
]
```

Muestro a continuación el código fuente. Es necesario *solo completar el código fuente del controller y del archivo de implementación del repository*.

```
// The files code:
```

```
package com.music.track.repository;

import com.music.track.model.Track;
```

```

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface TrackRepository extends JpaRepository<Track, Long> {
}

package com.music.track.service.impl;

import com.music.track.dto.TrackRequest;
import com.music.track.model.Track;
import com.music.track.repository.TrackRepository;
import com.music.track.service.TrackService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class TrackServiceImpl implements TrackService {
    private final TrackRepository trackRepository;

    @Autowired
    public TrackServiceImpl(TrackRepository trackRepository) {
        this.trackRepository = trackRepository;
    }

    @Override
    public Track createTrack(TrackRequest trackRequest) {
        return null;
    }

    @Override
    public List<Track> getAllTracks() {
        return null;
    }

    @Override
    public void deleteTrack(Long trackId) {
    }

    @Override
    public List<Track> sortedTracks() {
        return null;
    }
}

package com.music.track.service;

import com.music.track.dto.TrackRequest;
import com.music.track.model.Track;

import java.util.List;

public interface TrackService {
    Track createTrack(TrackRequest trackRequest);

    List<Track> getAllTracks();

    void deleteTrack(Long trackId);
    List<Track> sortedTracks();
}

package com.music.track.dto;

import java.util.Date;

public record TrackRequest(String title,
                           String albumName,
                           Date releaseDate,

```

```

        Integer playCount) {
    }

package com.music.track.model;

import jakarta.persistence.*;
import java.util.Date;

@Entity
@Table(name = "track")
public class Track {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "track_id")
    private Long id;
    @Column(name = "title")
    private String title;
    @Column(name = "album_name")
    private String albumName;
    @Column(name = "release_date")
    private Date releaseDate;
    @Column(name = "play_count")
    private Integer playCount;
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAlbumName() {
        return albumName;
    }

    public void setAlbumName(String albumName) {
        this.albumName = albumName;
    }

    public Date getReleaseDate() {
        return releaseDate;
    }

    public void setReleaseDate(Date releaseDate) {
        this.releaseDate = releaseDate;
    }

    public Integer getPlayCount() {
        return playCount;
    }

    public void setPlayCount(Integer playCount) {
        this.playCount = playCount;
    }
}

package com.music.track.controller;

import com.music.track.dto.TrackRequest;
import com.music.track.model.Track;

```

```

import com.music.track.service.TrackService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("music/platform/v1/tracks")
public class TrackController {

    private final TrackService trackService;
    @Autowired
    public TrackController(TrackService trackService) {
        this.trackService = trackService;
    }
    /**
     * Create a track
     * @param trackRequest
     * @return
     */
    @PostMapping()
    public ResponseEntity<Track> createTrack(@RequestBody TrackRequest trackRequest){
        return null;
    }
    /**
     * Get all tracks
     * @return
     */
    @GetMapping()
    public ResponseEntity<List<Track>> getAllTracks(){
        return null;
    }
    /**
     * Delete a track
     * @param trackId
     * @return
     */
    @DeleteMapping("/{trackId}")
    public ResponseEntity<Void> deleteTrack(@PathVariable Long trackId){
        return null;
    }

    /**
     * Get Track sorted
     * @return
     */
    @GetMapping("/sorted")
    public ResponseEntity<List<Track>> getTracksSorted() {
        return null;
    }
}

```

Spring Boot: API de información de tareas ejecutadas

En tu aplicación, necesitas permitir que los usuarios realicen las siguientes operaciones con las tareas:

- Crear, leer, actualizar y eliminar;
- Describir;
- Obtener todas;
- Describir todas.

Para cumplir con los requisitos, debes:

- Añadir las anotaciones faltantes a `TaskController`;

● Implementar todos los métodos y añadir las anotaciones necesarias para que estén disponibles a través de llamadas HTTP.

Consejos:

● Utiliza `TaskDto` para recibir información del usuario y enviársela de vuelta.

Requisitos. Lo que se pide es lo siguiente:

1. Crear una tarea:

- Petición
- URL: `/tasks`
- Método: `POST`
- Payload (ejemplo):

```
{
  "title" : "Task1",
  "description" : "Some description"
}
```

- Respuesta. Estado: 200
- Cuerpo (ejemplo): 13

Comentarios. Durante la creación, solo se pueden establecer el título y la descripción de una tarea. El cuerpo de la respuesta es el ID de la tarea recién creada.

2. Leer una tarea:

- Petición
- URL: `/tasks/{id}`
- Método: `GET`
- Respuesta
- Escenario 1: Tarea encontrada
- Estado: 200
- Cuerpo (ejemplo):

```
{
  "id": "1",
  "title": "Task 1",
  "description": "Some description",
  "status": "CREATED"
}
```

- Escenario 2: Tarea no encontrada
- Estado: 204

3. Actualizar una tarea:

- Petición
- URL: `/tasks/{id}`
- Método: `PUT`
- Payload (ejemplo):

```
{
  "title": "Task 1.1",
  "description": "New description",
}
```

```
    "status": "BLOCKED"
  }
```

- Respuesta

- Escenario 1: Tarea encontrada

Estado: 200

- Escenario 2: Tarea no encontrada

Estado: 204

- Escenario 3: Estado no válido proporcionado

Estado: 200

- Cuerpo:

Los estados disponibles son:

CREATED, APPROVED, REJECTED, BLOCKED, DONE.

4. Eliminar una tarea:

- Petición

URL: /tasks/{id}

- Método: DELETE

- Respuesta

- Escenario 1: Tarea encontrada

Estado: 200

- Escenario 2: Tarea no encontrada

5. Describir una tarea:

- Petición

Estado: 204

- URL: /tasks/describe/{id}

- Método: GET

- Respuesta

- Escenario 1: Tarea encontrada

Estado: 200

- Cuerpo (ejemplo):

```
[
  "Description"
  [13: Name] is: Some description"
]
```

- Escenario 2: Tarea no encontrada

Estado: 200

- Cuerpo (ejemplo):

["Task with id = 13 does not exist"]

6. Buscar todas las tareas:

- Petición

■ URL: /tasks

- Método: GET

- Respuesta

Estado: 200

■ Cuerpo (ejemplo):

```
[
  {
    "id": "1",
    "title": "Task 1",
    "description": "Some description",
    "status": "CREATED"
  },
  {
    "id": "2",
    "title": "Task2",
    "description": "Another description",
    "status": "CREATED"
  }
]
```

7. *Describir todas las tareas:*

■ Petición

■ URL: /tasks/describe

■ Método: GET

Respuesta

■ Estado: 200

■ Cuerpo (ejemplo):

```
[
  "Description of Task [1: Task 1] is: Some description",
  "Description of Task [2: Task 2] is: Another description"
]
```

Observaciones.

1. No es necesario preocuparse por la seguridad, las transacciones ni nada que no sean las peticiones REST.
2. Ya se ha inyectado una clase `TaskController` en `TaskRepository`. Lo que se busca es que escriba a lapiz y papel como es el código de `TaskController`.
3. Ya existe un modelo de datos implementado; puede utilizarlo, pero no modificarlo.
4. No es necesario preocuparse por la validación.
5. El código previo es el siguiente:

```
public interface TaskRepository extends CrudRepository<Task, Long> {}
```

```
// Task
@Entity
public class Task {
    @Id
    @GeneratedValue
    private Long id;

    private String title;
    private String description;
    private TaskStatus status = CREATED;

    public Task(String title) {
        this.title = title;
    }

    private Task() {}
}
```

```

    public void setTitle(String title) {
        this.title = title;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public void setTaskStatus(TaskStatus status) {
        this.status = status;
    }

    public Long getId() {
        return id;
    }

    public TaskDto toDto() {
        return new TaskDto(String.valueOf(id), title, description, status.name());
    }
}

// TaskStatus:
public enum TaskStatus {
    CREATED, APPROVED, REJECTED, BLOCKED, DONE
}

// TaskDto:
public class TaskDto {
    private String id;
    private String title;
    private String description;
    private String status;

    TaskDto(String id, String title, String description, String status) {
        this.id = id;
        this.title = title;
        this.description = description;
        this.status = status;
    }

    TaskDto() {}

    public String getId() {
        return id;
    }

    public String getTitle() {
        return title;
    }

    public String getDescription() {
        return description;
    }

    public String getStatus() {
        return status;
    }
}

```

Imagine que se usa la versión de Java y librerías disponibles:

- Java 21
- Apache Commons Lang 3:3.17 • Google Guava 33.4.8-jre
- Spring Boot Data JPA Starter 3.5.0
- Spring Boot Data Web Starter 3.5.0

Java Puro: Ejercicio simple de código fuente

Crear un código fuente que valide si un texto es palíndromo o no. Debe usarse la estructura solamente en programación funcional o reactiva exclusivamente (no imperativa o procedural).

Respuestas

Las respuestas a este ejercicio son las siguientes:

1. Exception Test

- **Análisis.** Revisando el código:

- `Person person = new Person(); person.sayHow(getHow());` -> Llama al método `sayHow(String)` por defecto de la interfaz `How`, el cual a su vez invoca a `sayHow(how, 1)`. Dado que `person` es una instancia de `Person`, se ejecuta el método sobreescrito en `Person`, imprimiendo "person".
- `Person employee = new Employee(); employee.sayHow(getHow());` -> Debido al polimorfismo, al ser una instancia de `Employee`, se invoca el método `sayHow(how, 1)` de la clase `Employee`, imprimiendo "employee".
- `User user = employee; user.sayHow(getHow());` -> Aunque la variable es de tipo `User`, la instancia real sigue siendo `Employee`. Por lo tanto, en tiempo de ejecución se vuelve a invocar el método de `Employee`, imprimiendo nuevamente "employee".

- **Opción correcta.** Es la opción **C**: `person, employee, employee`

2. Java Executor Service & Callable

- **Análisis.** El método `executor.submit(task)` acepta un objeto `Callable<Double>` y devuelve un objeto `Future<Double>`. Para obtener el resultado real (el valor `Double`), es necesario invocar el método `.get()`, manejando o propagando las excepciones correspondientes.
- **Opción correcta.** Es la opción **A**.

```
try {  
    return executor.submit(task).get();  
} catch (Exception ex) {  
    return null;  
}
```

3. Scoring System (SQL)

- **Análisis.** Para obtener los tres estudiantes con los puntajes más altos ordenados de forma descendente por `SCORE` y ascendente por `ID` en caso de empate, utilizando sintaxis compatible con MySQL (limitando a 3 registros).
- **Consulta SQL correcta.** Si usamos MySQL o MariaDb, la sentencia SQL sería esta:

```
SELECT t1.ID, t1.NAME  
FROM STUDENT t1  
ORDER BY t1.SCORE DESC, t1.ID ASC  
LIMIT 3;
```

4. Clases Estáticas.

- **Análisis.** Leyendo el enunciado del requerimiento, tenemos que crear una clase llamada `TransactionAnalyzer`. Vamos a usar el enfoque de la programación funcional en este desafío.
- **Opción correcta.** El código fuente es el siguiente:

```
package com.codility.solution;
```

```

import java.util.Arrays;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

public class TransactionAnalyzer {

    public static Object[] analyzeTransactions(int[] transactions) {
        int[] validTransactions = Optional.ofNullable(transactions)
            .orElseThrow(() -> new IllegalArgumentException("El arreglo de transacciones no
puede ser nulo."));

        int totalSum = Arrays.stream(validTransactions).sum();
        int maxTransaction = Arrays.stream(validTransactions).max().orElse(0);
        long countAboveThousand = Arrays.stream(validTransactions)
            .filter(t -> t > 1000).count();

        List<Integer> uniqueSortedTransactions = Arrays.stream(validTransactions)
            .boxed()
            .distinct()
            .sorted()
            .collect(Collectors.toList());

        return new Object[] {
            totalSum, maxTransaction, countAboveThousand, uniqueSortedTransactions };
    }
}

```

5. Java Classes

- **Análisis.** Las clases miembro estáticas (*static member classes*) son esencialmente clases anidadas estáticas que se comportan como cualquier otro miembro de nivel superior de una clase, por lo que sí pueden contener métodos no estáticos.
- **Opción correcta.** *Static member classes can contain non-static methods.* Es la opción **D**.

6. Static Analysis 3

- **Análisis.** Revisando el código:
 - Se pasa por argumento de propiedad del sistema `-DargProp="dog."` (nota que el valor incluye un punto al final: `"dog."`).
 - `p.getProperty("argProp")` evalúa a `"dog."`.
 - `p.getProperty("pirate")` evalúa a `"scurvy"` (definido previamente en el código).
 - Al concatenarlos (`s += p.getProperty("pirate")`), el resultado es `"dog. scurvy"` o similar, pero revisando cuidadosamente la estructura del código proporcionado, la propiedad leída es `"dog."` unida directamente con espacio o concatenación de cadenas. Sin embargo, analizando las opciones disponibles: `dog`, `scurvy`, `null`, `scurvy`, `scurvy dog`, `scurvy null`. Nota que el código base contiene un pequeño error sintáctico de inicialización directa de variables de instancia con sentencias ejecutables fuera de un bloque o método, pero asumiendo la lógica de propiedades: `argProp` trae `"dog."` y `pirate` trae `"scurvy"`.
- **Opción correcta.** La opción es **A**, puesto que muestra la salida `dog, scurvy`.

7. Spring Boot: API de información de tareas ejecutadas

Teniendo en cuenta el repository, dto y el objeto enum, tenemos que completar el código fuente de TaskController. La solución es la siguiente:

```

package com.codility.rest;

```

```

import com.codility.domain.*;
import com.google.common.collect.Streams;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.*;
import java.util.stream.*;

import static java.util.Arrays.asList;
import static java.util.stream.Collectors.toList;

@RestController
@RequestMapping("/tasks")
public class TaskController {
    private final TaskRepository repository;

    @Autowired
    public TaskController(TaskRepository repository) {
        this.repository = repository;
    }

    @PostMapping
    public ResponseEntity<Long> createTask(@RequestBody TaskDto taskDto) {
        Task task = new Task(taskDto.getTitle());
        task.setDescription(taskDto.getDescription());
        Task savedTask = repository.save(task);
        return ResponseEntity.ok(savedTask.getId());
    }

    @GetMapping("/{id}")
    public ResponseEntity<TaskDto> getTaskById(@PathVariable Long id) {
        Optional<Task> task = repository.findById(id);
        if (task.isPresent()) {
            return ResponseEntity.ok(task.get().toDto());
        }
        return ResponseEntity.status(204).build();
    }

    @PutMapping("/{id}")
    public ResponseEntity<Object> updateTask(@PathVariable Long id, @RequestBody TaskDto
taskDto) {
        Optional<Task> optionalTask = repository.findById(id);
        if (optionalTask.isEmpty()) {
            return ResponseEntity.status(204).build();
        }

        TaskStatus status;
        try {
            status = TaskStatus.valueOf(taskDto.getStatus());
        } catch (IllegalArgumentException | NullPointerException e) {
            // Asegurar el formato exacto de una sola línea con los espacios requeridos
            return ResponseEntity.ok("Available statuses are:
CREATED, APPROVED, REJECTED, BLOCKED, DONE.");
        }

        Task task = optionalTask.get();

        // Actualizar solo los campos que no sean nulos
        if (taskDto.getTitle() != null) {
            task.setTitle(taskDto.getTitle());

```

```

    }

    if (taskDto.getDescription() != null) {
        task.setDescription(taskDto.getDescription());
    }

    task.setTaskStatus(status);
    repository.save(task);

    return ResponseEntity.ok().build();
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteTask(@PathVariable Long id) {
    if (repository.existsById(id)) {
        repository.deleteById(id);
        return ResponseEntity.ok().build();
    }
    return ResponseEntity.status(204).build();
}

@GetMapping("/describe/{id}")
public ResponseEntity<List<String>> describeTask(@PathVariable Long id) {
    Optional<Task> task = repository.findById(id);
    List<String> response = new ArrayList<>();
    if (task.isPresent()) {
        Task t = task.get();
        response.add("Description of Task [" + t.getId() + ": " + t.toDto().getTitle() + "]
is: " + t.toDto().getDescription());
    } else {
        response.add("Task with id = " + id + " does not exist");
    }
    return ResponseEntity.ok(response);
}

@GetMapping
public ResponseEntity<List<TaskDto>> getAllTasks() {
    List<TaskDto> tasks = Streams.stream(repository.findAll())
        .map(Task::toDto)
        .collect(toList());
    return ResponseEntity.ok(tasks);
}

@GetMapping("/describe")
public ResponseEntity<List<String>> describeAllTasks() {
    List<String> descriptions = Streams.stream(repository.findAll())
        .map(t -> "Description of Task [" + t.getId() + ": " + t.toDto().getTitle() +
"] is: " + t.toDto().getDescription())
        .collect(toList());
    return ResponseEntity.ok(descriptions);
}
}

```

8. Java Puro: Ejercicio simple de código fuente

En esta pregunta se requiere el desarrollo de una función que valide una frase si es un palíndromo o no. Usaremos la programación funcional basada en Streams. La solución es la siguiente:

```

import lombok.extern.slf4j.Slf4j;
import lombok.RequiredArgsConstructor;

```

```

import java.util.Optional;
import java.text.Normalizer;
import java.util.stream.IntStream;
import java.util.function.Predicate;

import org.springframework.stereotype.Service;

@Slf4j
@Service
@RequiredArgsConstructor
public class PalindromeService {

    public Predicate<String> validarPalindromoFuncional() {
        return finalText -> Optional.ofNullable(finalText)
            .map(textValue -> Normalizer.normalize(textValue.toLowerCase(), Normalizer.Form.NFD)
                .replaceAll("[\\p{InCombiningDiacriticalMarks}]", "")
                .replaceAll("[^a-zA-Z0-9]", ""))
            .filter(cleanText -> !cleanText.isEmpty())
            .map(cleanText -> IntStream.range(0, cleanText.length() / 2)
                .allMatch(i -> cleanText.charAt(i) == cleanText.charAt(cleanText.length() - 1
- i)))
            .orElse(false);
    }

    public Predicate<String> validarPalindromoImperativo() {
        return texto -> {
            if (texto == null || texto.isBlank()) {
                return false;
            }

            // 1. Convertir a minúsculas
            String textoLimpio = texto.toLowerCase();

            // 2. Eliminar acentos y tildes (Ej: 'á' pasa a 'a')
            textoLimpio = Normalizer.normalize(textoLimpio, Normalizer.Form.NFD);
            textoLimpio = textoLimpio.replaceAll("[\\p{InCombiningDiacriticalMarks}]", "");

            // 3. Eliminar espacios, puntos, comas y signos de interrogación
            textoLimpio = textoLimpio.replaceAll("[^a-zA-Z0-9]", "");

            // 4. Verificar con dos punteros (inicio y fin)
            int izquierda = 0;
            int derecha = textoLimpio.length() - 1;

            while (izquierda < derecha) {
                if (textoLimpio.charAt(izquierda) != textoLimpio.charAt(derecha)) {
                    return false;
                }
                izquierda++;
                derecha--;
            }

            return true;
        };
    }
}

```